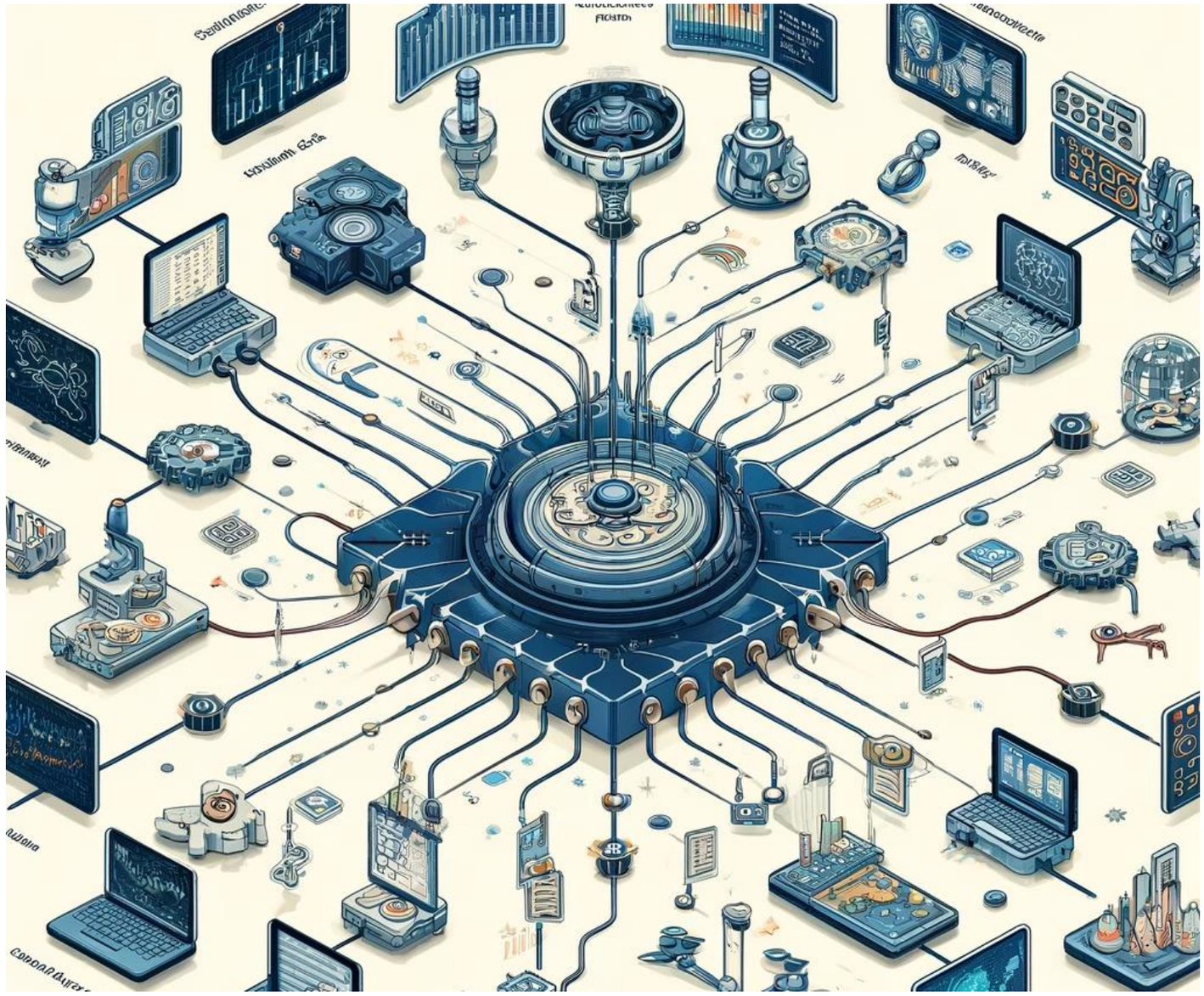
Exploring Scientific Workflows with CWL and dispel4py

Day 2- Module 3

- Dr. Rosa Filgueira
- Chancellor's Fellow AI & DS
- University of Edinburgh, EPCC
- r.filgueira@epcc.ed.ac.uk



dispel4py Installation

We have several options to install dispel4py! It uses **Python 3.10 .**

1) Follow the instructions specified in our StreamingFlow/d4py repository

Via pip

1. `conda create --name stream-d4py_env python=3.10`
2. `conda activate stream-d4py_env`
3. `conda install -c conda-forge mpi4py mpich` OR `pip install mpi4py` (Linux)
4. `pip install stream-d4py`

Via cloning this repo

1. `conda create --name stream-d4py_env python=3.10`
2. `conda activate stream-d4py_env`
3. `https://github.com/StreamingFlow/stream-d4py.git`
4. `cd dispel4py`
5. `conda install -c conda-forge mpi4py mpich` OR `pip install mpi4py` (Linux)
6. `python setup.py install`

dispel4py Installation

Use directly our [Google Notebook Colab](), which installs dispel4py

Testing_dispel4py_2_0.ipynb

File Edit View Insert Runtime Tools Help

Table of contents

dispel4py
Python Version
Getting Started with dispel4py
Part I- Simple Examples
Word_Count dispel4py Workflow
Part II - Real Use Cases
Covid Workflow
Sensor Workflow
Postprocess Sensor_output.txt

+ Section

+ Code + Text Copy to Drive

dispel4py

dispel4py is a free and open-source Python library for describing abstract stream-based workflows for distributed data-intensive applications. It enables users to focus on their scientific methods, avoiding distracting details and retaining flexibility over the computing infrastructure they use. It delivers mappings to diverse computing infrastructures, including cloud technologies, HPC architectures and specialised data-intensive machines, to move seamlessly into production with large-scale data loads. The dispel4py system maps workflows dynamically onto multiple enactment systems, and supports parallel processing on distributed memory systems with MPI and shared memory systems with multiprocessing, without users having to modify their workflows.

Python Version

This version of dispel4py has been tested with Python 3.10

For earlier versions of dispel4py compatible with Python < `3.10 ( 2.7.5, 2.7.2, 2.6.6 and Python 3.4.3, 3.6, 3.7)` we recommend to go here.

Getting Started with dispel4py

Our journey begins with the installation of dispel4py, ensuring you have the necessary tools to dive into data processing. We present two distinct approaches to install dispel4py:

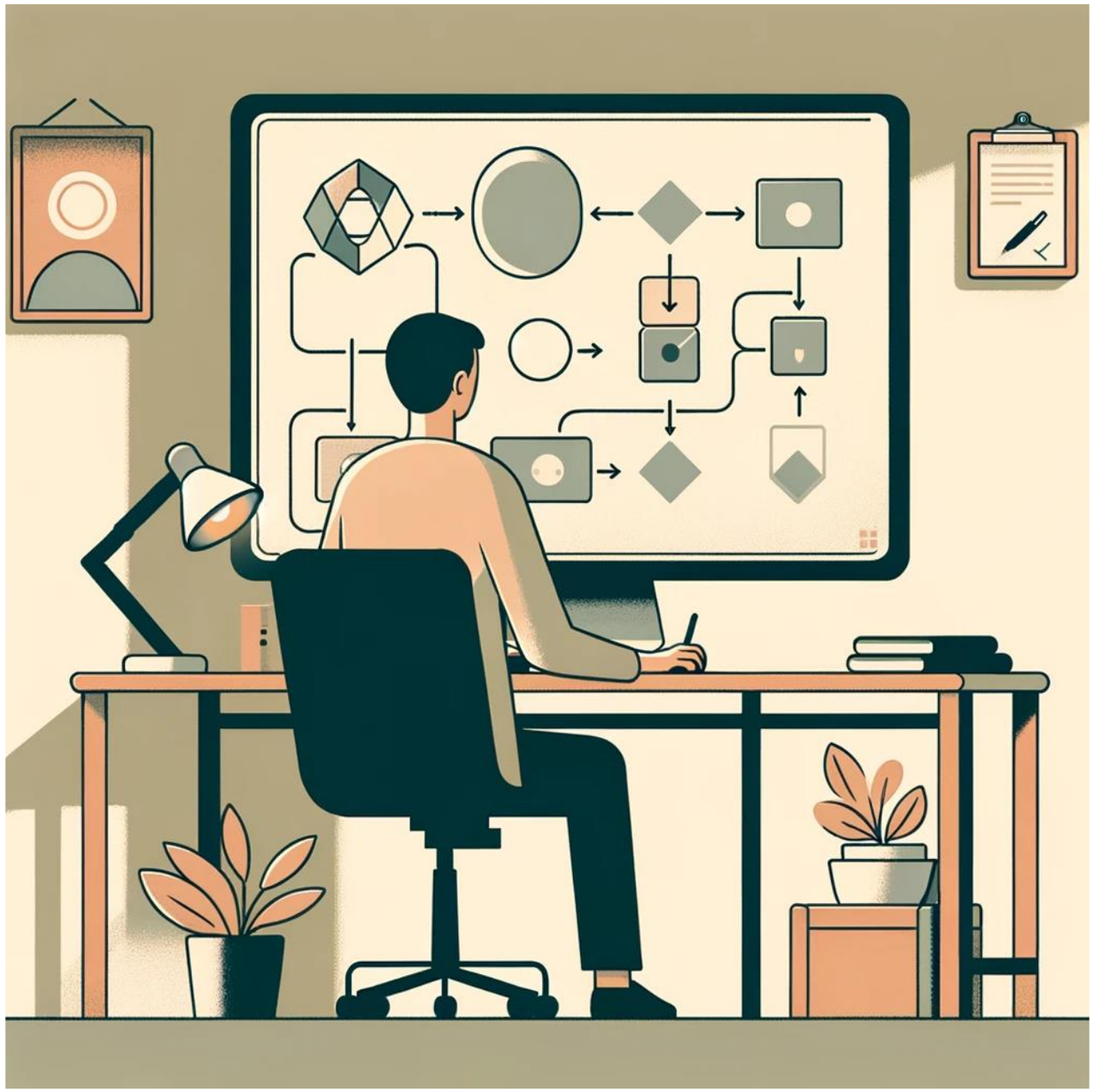
- Version 1: Direct installation using pip, suitable for quickly adding dispel4py and its stream-d4py companion to your Python environment.
- Version 2: For those preferring the latest versions or contributing to development, cloning from the GitHub repository and manual installation offers the cutting edge of dispel4py functionalities.

```
[ ]
#Version 1:
!pip install mpi4py
!pip install stream-d4py

# Version 2:
#!git clone https://github.com/StreamingFlow/d4py.git
#%cd d4py
#!pip install mpi4py
#!python setup.py install
```

```
Collecting mpi4py
  Downloading mpi4py-3.1.5.tar.gz (2.5 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 28.2 MB/s eta 0:00:00
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Building wheels for collected packages: mpi4py
  Building wheel for mpi4py (pyproject.toml) ... done
  Created wheel for mpi4py: filename=mpi4py-3.1.5-cp310-cp310-linux_x86_64.whl size=2746507 sha256=b4020eb5e2e9b75c7729b9ae881f0e33596fe7c90e6495b17bad8b015f107064
  Stored in directory: /root/.cache/pip/wheels/18/2b/7f/c852523089e9182b45fca50ff56f49a51eeb6284fd25a66713
Successfully built mpi4py
Installing collected packages: mpi4py
Successfully installed mpi4py-3.1.5
Collecting stream-d4py
  Downloading stream_d4py-2.9.1-py3-none-any.whl (121 kB)
```

dispel4py Basic Concepts

- What is dispel4py
- What is a stream
- What is a processing element (PE)
- What I need for constructing a workflow
- Learning dispel4py by an example
- How to Implement a PE



What is dispel4py

- dispel4py for distributed **data-intensive applications**
- It enables **abstract description of methods**
- dispel4py maps to **multiple enactment** systems

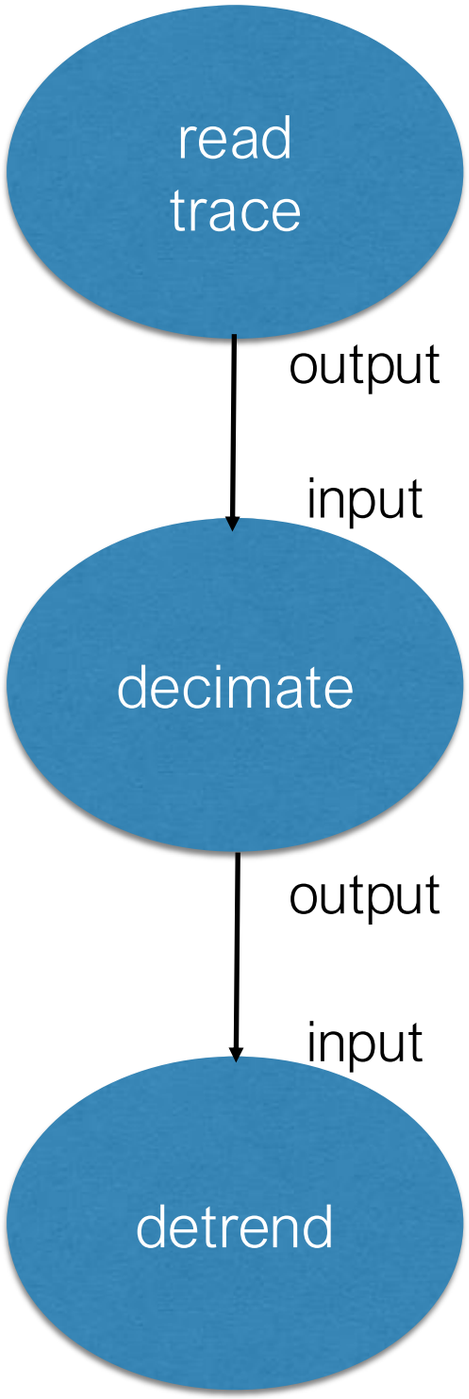
- Applications **scale automatically**
  - exploiting parallel processing, clusters, and clouds

- dispel4py is **dataflow-oriented**
  - rather than control-oriented
  - no explicit specification of data movement
  - light-weight composition of data operations

What is a data stream

- A **stream** is a sequence of data units:
  - from external source
  - between data operations - Processing Elements (PEs)
  - to external destination
- Flow of input or output data between PEs
- Processes data from a source and delivers data to one or more destinations

What is a processing element (PE)



- Computational activity encapsulates
  - algorithm
  - services
  - data transformation processes
- Basic computational elements of dispel4py workflows
- PEs have:
  - inputs & outputs
  - computational activity.
- PEs are connected by streams
  - saves computational costs

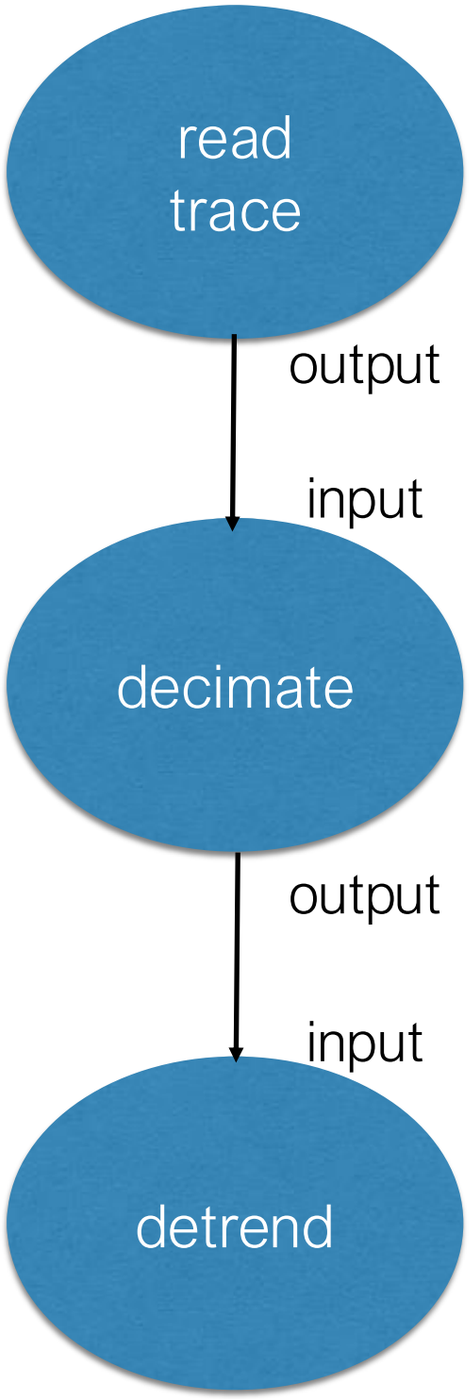
What is a graph

- How the PEs are **connected**
- How data is **streamed**
- The **topology** of the data flow
- **No limitations** on the type of graphs
- Graphs are **DAGs** in dispel4py

What I need for constructing a dispel4py workflow

- You **only** have to **implement PEs** (in Python) and connect them:
  - Learn how to implement PEs.
  - Learn how to connect them.

Learning dispel4py by an example



- dispel4py workflow that reads a trace
- decimate the trace → **reduces the sampling rate of the trace**
- detrend the trace → **removes the artificial drift caused by the instrument or recording**

```
Before decimation:

Time (s):       0.0   0.1   0.2   0.3   0.4   0.5   0.6   0.7   0.8   0.9
Amplitude:     2.1   2.4   2.0   2.7   2.3   2.9   2.6   3.1   2.8   3.4

After decimation:

Time (s):       0.0         0.2         0.4         0.6         0.8
Amplitude:     2.1         2.0         2.3         2.6         2.8
```

10 samples per second → decimate by 2 → 5 samples per record (faster to process).

```
Normal signal:

 1      /\      /\
 0  ___/  \____/  \___
-1
```

```
Signal with drift:

 5              /\
 4         /\   /  \
 3    /\  /  \_/
 2 ___/  \/
 1
 0
```

```
After detrending:

 1      /\      /\
 0  ___/  \____/  \___
-1
```

Detrend flattens the signal again.

How to implement a PE

- Each PE specifies:
  - input & output connections
  - computational activity for processing data units
  - implement the “_process” method.

Types of PEs

Type	Inputs	Outputs	When to use it
GenericPE	*n* inputs	*m* outputs	many inputs and/or many outputs
IterativePE	1 input named 'input'	1 output named 'output'	process one and produce one data unit in each iteration
Consumer PE	1 input named 'input'	no output	no output and one input ; usually last PE of graph
ProducerPE	no input	1 output named 'output'	no inputs and one output; usually the root in a graph
Simple FunctionPE	1 input named 'input'	1 output named 'output'	only implement _process method; it can not store state between calls
create_iterative_chain	1 input named 'input'	1 output named 'output'	pipeline of functions processing sequentially; creates a composite PE

GenericPE example

```
from dispel4py.core import GenericPE

class StreamAndStatsProducer(GenericPE):
    def __init__(self):
        GenericPE.__init__(self)
        self._add_input('input')
        self._add_output('output')
        self._add_output('output_stats')

    def process(self, inputs):
        data = inputs['input']
        filename = data
        st = read(filename)
       return {'output': st, 'output_stats': st[0].stats}
```

This PE reads a file that contains seismological traces and returns two outputs: obspy stream and metadata.

What we have learnt:

- We can add several outputs with different names
- The process method gets values from the input streams
- The process method returns both streams

IterativePE example

```
from dispel4py.base import IterativePE
from obspy.core import read

class ReadTrace(IterativePE):
    def __init__(self):
        IterativePE.__init__(self)
    def _process(self, data):
        filename = data
        st = read(filename)
        return st
```

This PE receives a filename (‘input’), reads the obspy trace from the file, and writes it to the output (‘output’)

What we have learnt:

- We don’t need to specify the input and output
- The parameter to the _process method is the data (the filename)
- _process returns the value that is written to the output stream

SimpleFunctionPE example

```
from dispel4py.base import SimpleFunctionPE

def decimate(st, sps):
    st.decimate(int(st[0].stats.sampling_rate/sps))
    return st

decimate = SimpleFunctionPE(decimate, {‘sps’: 4})
```

This PE will emit a decimated trace.

What we have learnt:

- Only implement the processing function
- The easiest but the most restrictive way
- 1 input called ‘input’, 1 output called ‘output’.

ConsumerPE example

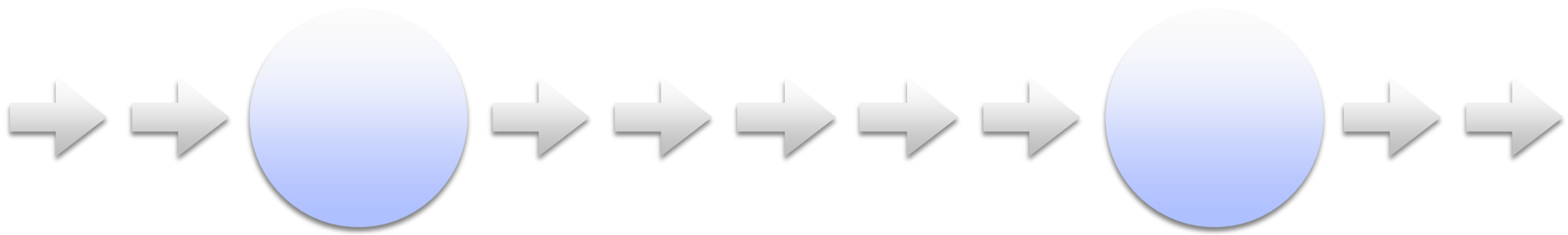
```
from dispel4py.base import ConsumerPE

class Detrend(ConsumerPE):
    def __init__(self):
        ConsumerPE.__init__(self)
    def _process(self, st):
        st.detrend('simple')
        st.write(st[0].getId() + '.mseed', 'MSEED')
```

This PE receives one decimated trace, applies detrend and writes it to a MSEED file.

What we have learnt:

- We don't need to specify the input
- It does not return any output.

How to connect PEs: What does it mean?



- PEs process a small amount of data at a time
- Data need not be explicitly stored
- PEs may store a small amount of result data (e.g. stacking) or big amount (if you have the resources)

How to connect PEs:

Create the PEs

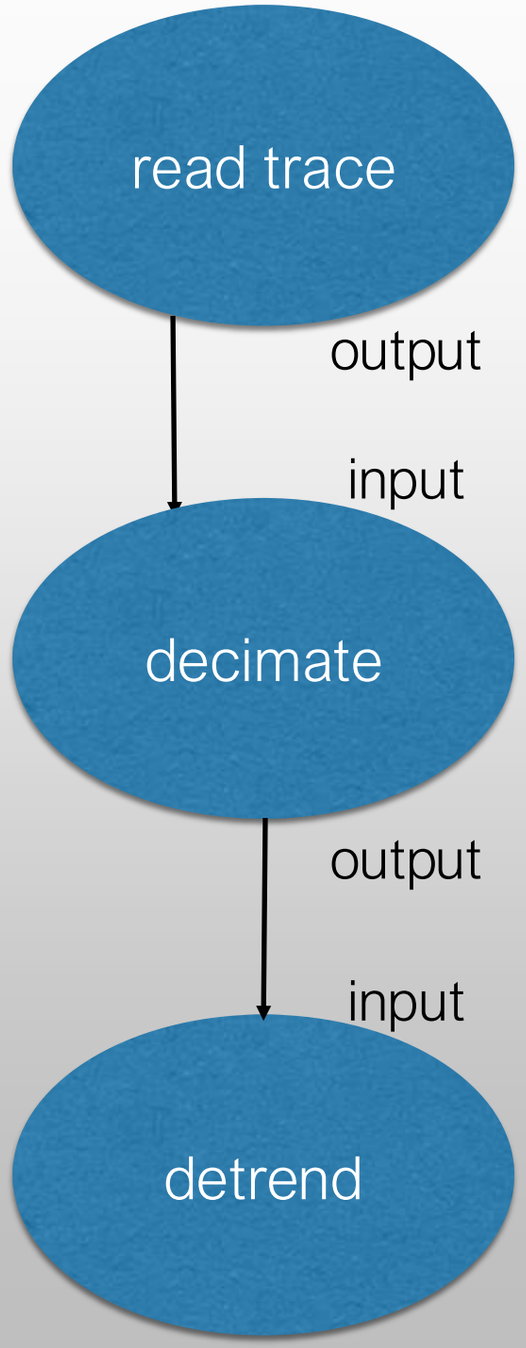
```
readtrace = ReadTrace()
decimate = SimpleFunctionPE(decimate, {‘sps’: 4})
detrend = Detrend()
```

Create the graph and connect the PEs

```
from dispel4py.workflow_graph import WorkflowGraph

graph = WorkflowGraph()
graph.connect(readtrace, 'output', decimate, ‘input')
graph.connect(decimate, 'output', detrend, 'input')
```

Example- Summary

```
from dispel4py.base import IterativePE, ConsumerPE, SimpleFunctionPE
from dispel4py.workflow_graph import WorkflowGraph
from obspy.core import read

class ReadTrace(IterativePE):
    def __init__(self):
        IterativePE.__init__(self)
    def _process(self, data):
        filename = data
        st = read(filename)
        return st

def decimate(st, sps):
    st.decimate(int(st[0].stats.sampling_rate/sps))
    return st

class Detrend(ConsumerPE):
    def __init__(self):
        ConsumerPE.__init__(self)
    def _process(self, st):
        st.detrend('simple')
     st.write(st[0].getId() + '.mseed', 'MSEED')

readtrace = ReadTrace()
decimate = SimpleFunctionPE(decimate, {'sps': 4})
detrend = Detrend()

graph = WorkflowGraph()
graph.connect(readtrace, 'output', decimate, 'input')
graph.connect(decimate, 'output', detrend, 'input')
```



Example II – Can you guess what this workflow does ?

```python
from dispel4py.base import ProducerPE, IterativePE, ConsumerPE
from dispel4py.workflow_graph import WorkflowGraph
import random
from easydict import EasyDict as edict




class NumberProducer(ProducerPE):
    def __init__(self):
        ProducerPE.__init__(self)

    def _process(self, inputs):
        # this PE produces one input
        result= random.randint(1, 1000)
        return result

class IsPrime(IterativePE):
    def __init__(self):
        IterativePE.__init__(self)
    def _process(self, num):
        # this PE consumes one input and produces one output

        if all(num % i != 0 for i in range(2, num)):
            return num

class PrintPrime(ConsumerPE):
    def __init__(self):
        ConsumerPE.__init__(self)
    def _process(self, num):
        # this PE consumes one input
        print("the num %s is prime\n" % num, end="")


producer = NumberProducer()
isprime = IsPrime()
printprime = PrintPrime()

graph = WorkflowGraph()
graph.connect(producer, 'output', isprime, 'input')
graph.connect(isprime, 'output', printprime, 'input')
```

Example II – Can you guess what this workflow does ?

This workflow demonstrates the processing of random number streams using various Processing Elements (PEs).

- It begins with a *NumberProducer* PE generating random integers between 1 and 1000.
- These numbers are then routed to *IsPrime* PE, which categorizes them as prime or not, and only outputs the prime numbers to the next PE.
- Finally, the *PrintPrime* PE **receives only the prime numbers** and prints those out.